

MEDIATEK

MT8676 Hypervisor I2C User Manual

版本: 1.2
出版日期: 2025-03-13

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2024-11-15	张厚松	正式版
1.1	2024-12-27	王彧	增加章节 1.3.3 中断配置说明和 1.3.4 虚拟化配置方法
1.2	2025-03-17	王舜昌	增加 章节 1.3.5 节点配置 和 1.3.6 特殊应用需求的配置

目录

版本记录 2

目录 3

图片目录 4

表格目录 4

1 I2C 5

 1.1 概述 5

 1.1.1 简单介绍 5

 1.1.2 名词解释 5

 1.2 架构/流程概述 5

 1.2.1 I2C 介绍 6

 1.2.2 MT8676 I2C 功能 6

 1.2.3 I2C 传输格式 6

 1.3 配置/客制化指南 7

 1.3.1 设备树 7

 1.3.2 频率 7

 1.3.3 中断配置说明 8

 1.3.4 虚拟化配置方法 8

 1.3.5 节点配置 9

 1.3.6 特殊应用需求的配置 9

 1.4 常见问题/故障排除 10

 1.4.1 I2C 问题调试方法 10

 1.4.2 DTS 配置 10

 1.4.3 确认 GPIO 模式 10

 1.4.4 测量波形 10

 1.4.5 如何打印 I2C 寄存器信息 11

 1.4.6 寻求联发科技帮助 11

附件一 附加条款 12

图片目录

图 1-1. I2C Master 和 I2C Slave 之间的引脚连接.....5

图 1-2. I2C 传输格式6

表格目录

表 1-1. 名词解释.....5

1 I2C

1.1 概述

1.1.1 简单介绍

本章节介绍 MT8676 I2C 控制器的硬件、软件及其功能。

1.1.2 名词解释

表 1-1. 名词解释

缩写	全称	释义
DMA	Direct Memory Access	直接内存访问
FIFO	First Input First Output	先进先出
FM	Fast Mode	快速模式
FM+	Fast Mode Plus	快速模式+
GPIO	General Purpose Inputs-Outputs	通用输入输出
HS mode	High Speed Mode	高速模式
I/O	Input/Output	输入/输出
I2C	Inter-IC	互连集成电路
SCL	Serial Clock Line	串行时钟线
SDA	Serial Data Line	串行数据线
SM	Standard Mode	标准模式

1.2 架构/流程概述



图 1-1. I2C Master 和 I2C Slave 之间的引脚连接

1.2.1 I2C 介绍

I2C 控制器是一种双向、双线串行接口，利用串行时钟线（SCL）和串行数据线（SDA）信号。这些信号可由 I2C 中的主设备或从设备驱动。此通用控制器支持主设备角色并符合 I2C 规范。

1.2.2 MT8676 I2C 功能

- Start, repeated start and stop conditions generation
- Bus detection
- Acknowledge bit generation and detection
- 7-bit/10-bit addressing
- Clock stretching
- Active drive/wired-and I/O configuration
- I2C has four operating speeds:
 - I2C Standard mode (SM) with speed up to 100 kbit/s
 - I2C Fast mode (FM) with speed up to 400 kbit/s
 - I2C Fast mode Plus (FM+) with speed up to 1 Mbit/s
 - I2C High Speed (HS) mode with speed up to 3.4 Mbit/s
- Adjustable clock speed for SM/FM/FM+/HS mode operation
- FIFO mode and DMA mode
- Multiple transfer formats:
 - Multi-write per transfer
 - Multi-read per transfer
 - Multi-transfer per transaction
 - Combined format transfer with length change capability
 - Combined format transfer with direction change capability
 - Multi-transfer with a repeated start condition

1.2.3 I2C 传输格式

图 1-2 说明了 I2C 的 SM/FM/FM+ 使用的基本传输格式。

首先，主机发送一个启动条件。随后，主机发送它打算与之通信的 I2C 从属设备的 7 位静态地址。一旦从属设备响应寻址，主机就会发送/接收数据。数据传输完成后，主机发送停止条件，总线返回到自由状态。

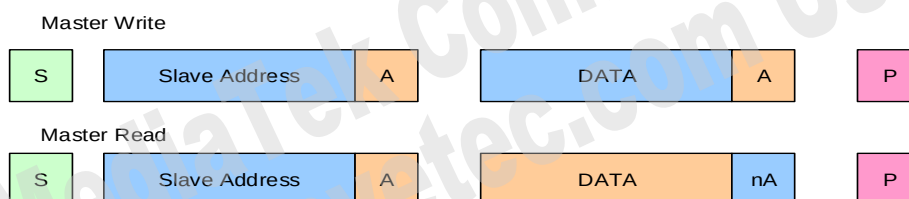


图 1-2. I2C 传输格式

1.3 配置/客制化指南

1.3.1 设备树

源代码路径: src/kernel/linux/v-xx/arch/arm64/boot/dts/mediatek/xxxx.dts

- 添加 I2C pin 以设置 I2C pinmux

```
&pio {
    i2c2_pins: i2c2-default {
        pins-bus {
            pinmux = <PINMUX_GPIO188__FUNC_SCL2>,
                    <PINMUX_GPIO189__FUNC_SDA2>;
            bias-pull-up = <MTK_PULL_SET_RSEL_111>;
        };
    };
};
```

- 在 I2C 总线上添加从设备

```
&i2c2 {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&i2c2_pins>;

    nxp_eusb2_repeater_p2: nxp-eusb2-repeater-p2@4f {
        compatible = "mtk,nxp-eusb2-repeater";
        pinctrl-names = "enable", "disable";
        pinctrl-0 = <&ptn3222_enable_p2>;
        pinctrl-1 = <&ptn3222_disable_p2>;
        reg = <0x4f>;
        #phy-cells = <0>;
        status = "okay";
    };
};
```

1.3.2 频率

I2C 控制器支持 100K、400K、1M 等频率。

您可以通过 DTS 设置 `clock-frequency = <xxxxxx>;`;

```
&i2c2 {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&i2c2_pins>;
    clock-frequency = <400000>;
};
```

1.3.3 中断配置说明

MT8676 Hypervisor I2C 若是以中断 passthrough 的方式使用，则中断默认送到 Yocto 一侧，如果将中断配置到 Android 侧，则 Yocto 侧无法使用中断。如果您希望以这种方式修改 I2C 中断，请向联发科技寻求协助。

1.3.4 虚拟化配置方法

源代码路径：src/kernel/linux/v-xx/arch/arm64/boot/dts/mediatek/xxxx.dts

- 虚拟化 I2C 总线节点

```
i2c0_virtio:i2c0-virtio{
    compatible = "mediatek,virtio-i2c";
    id = <0>;
    status = "disabled";
};

i2c1_virtio:i2c1-virtio{
    compatible = "mediatek,virtio-i2c";
    id = <0>;
    status = "disabled";
};

i2c2_virtio:i2c2-virtio{
    compatible = "mediatek,virtio-i2c";
    id = <0>;
    status = "disabled";
};
```

- 在虚拟化 I2C 总线上添加从设备

```
&i2c2_virtio {
    status = "okay";

    nxp_eusb2_repeater_p2: nxp-eusb2-repeater-p2@4f {
        compatible = "mtk,nxp-eusb2-repeater";
        pinctrl-names = "enable", "disable";
        pinctrl-0 = <&ptn3222_enable_p2>;
        pinctrl-1 = <&ptn3222_disable_p2>;
        reg = <0x4f>;
        #phy-cells = <0>;
        status = "okay";
    };
};
```

1.3.5 节点配置

默认需要将 master 节点的 status 配置成 “disabled”

```
i2cX: i2c@xxxx {  
    compatible = "mediatek,mt6897-i2c";  
    ....  
    status = "disabled";  
};
```

使用哪个项目，就将其子节点的 status 配置成 “okay”

```
&i2cX {  
    status = "okay";  
};
```

1.3.6 特殊应用需求的配置

有功耗需求，需要将 I2C 进 suspend 配置成低电平时相关方法如下：

```
&i2cX {  
    status = "okay";  
    pinctrl-names = "default", "sleep";  
    pinctrl-0 = <&i2cX_default>;  
    pinctrl-1 = <&i2cX_sleep>;  
};
```

```
&pio {  
    i2cX_default: i2cX-pins-default {  
        pins-bus {  
            pinmux = <PINMUX_GPIOxxx_FUNC_SCLX>,  
                    <PINMUX_GPIOxxx_FUNC_SDAX>;  
            bias-pull-up = <MTK_PULL_SET_RSEL_111>;  
        };  
    };  
  
    i2c3_sleep: i2c3-pins-sleep {  
        pins-bus {  
            pinmux = <PINMUX_GPIOxxx_FUNC_SCLX>,  
                    <PINMUX_GPIOxxx_FUNC_SDAX>;  
            bias-pull-down = <MTK_PULL_SET_RSEL_111>;  
        };  
    };  
};
```

(此方法只针对使用内部上拉的配置方法，外部上拉需要通过控制外部上拉的供电实现此功能)

1.4 常见问题/故障排除

1.4.1 I2C 问题调试方法

大多数遇到的 I2C 问题都可先尝试通过如下方式调试：

- 检查 salve device 初始化，供电是否正常
- 检查 DTS 配置、GPIO 模式属性是否正确
- 检查 I2C 寄存器信息是否正确（可以提供 log 给 RD）

1.4.2 DTS 配置

请按照前文的提供的信息，检查 DTS 配置是否正确。

1.4.3 确认 GPIO 模式

内核里查看引脚的状态：

输入以下命令：

```
# cd /sys
# find -name mt_gpio
# cat mt_gpio //
```

要输入绝对路径：

例如：

```
# cat /sys/devices/platform/soc/1000b000.pinctrl/mt_gpio
PIN: [MODE] [DIR] [DOUT] [DIN] [PULL_EN] [PULL_SEL] [IES] [SMT] [DRIVE] ( [R1] [R0] )
0: 0 0 0 0 1 0 1 0 0
1: 0 0 0 0 1 0 1 0 0
2: 0 1 1 1 1 0 1 0 0
3: 6 0 0 0 1 0 1 0 0
```

1.4.4 测量波形

使用示波器测量 I2C 波形是否符合预期。

1.4.5 如何打印 I2C 寄存器信息

在 `driver/i2c/busses/i2c-mt65xx.c` 中添加如下内容：

```
static void i2c_dump_register(struct mtk_i2c *i2c)
{
    dev_dbg(i2c->dev, "SLAVE_ADDR: 0x%x, INTR_MASK: 0x%x\n",
            mtk_i2c_readw(i2c, OFFSET_SLAVE_ADDR),
            mtk_i2c_readw(i2c, OFFSET_INTR_MASK));
    dev_dbg(i2c->dev, "INTR_STAT: 0x%x, CONTROL: 0x%x\n",
            mtk_i2c_readw(i2c, OFFSET_INTR_STAT),
            mtk_i2c_readw(i2c, OFFSET_CONTROL));
}
```

将 `dev_dbg` 改为 `dev_err`。

1.4.6 寻求联发科技帮助

若经过前文的排查仍无法解决您的问题，向联发科技寻求帮助时顺便提供前文提到的包含 I2C 寄存器信息的相关日志、波形图、DTS、`cat mt_gpio` 等信息。

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿責任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不应承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。